Terraform Cloud Beginners Guide Remote State and Pipeline

Managing Terraform state locally works for small experiments, but breaks down quickly when working in teams. Terraform Cloud gives you free remote state storage, secret management, and automatic CI/CD pipelines with zero maintenance.

What you'll learn:

1. Setting Up Free Remote State
2. Connecting Terminal to Terraform Cloud
3. Storing Secrets with Cloud Variables
4. Automating Deployments with GitHub

Aman Raj Singh
Cloud Operations Engineer @ iManage
linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Setting Up Free Remote State

Instead of saving terraform.tfstate on your laptop where it can be deleted or leaked, you can send it to Terraform Cloud. Simply add the cloud block inside your terraform block in main.tf. Running terraform init automatically migrates your local state file to secure cloud storage without losing any data.

```
terraform {
  cloud {
    organization = "my-company-name"
    workspaces {
      name = "my-web-app-dev"
    }
  }
}
```

2

TIP 2 OF 4

Connecting Terminal to Terraform Cloud

To let your local terminal communicate safely with your cloud account, you need an API token. Running terraform login opens a browser window where you generate and copy a personal access token. Paste the token into your prompt to authenticate your terminal session instantly.

```
$terraform login
```

3

TIP 3 OF 4

Storing Secrets with Cloud Variables

Never hardcode AWS access keys or database passwords inside your configuration files. In Terraform Cloud, navigate to Workspace > Variables and add them as Environment Variables. Prefixing variables with TF_VAR_ lets Terraform inject them into your runs safely without committing plain text secrets.

```
$# Added in Terraform Cloud UI (Workspace Environment Variables):
 AWS_ACCESS_KEY_ID = "AKIAIOSFODNN7EXAMPLE"
 AWS_SECRET_ACCESS_KEY = "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"
```

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

Automating Deployments with GitHub

You can connect your GitHub repository directly to a Terraform Cloud workspace. Every time a developer opens a Pull Request, Terraform Cloud automatically runs terraform plan to show expected changes. Once merged into main, it executes terraform apply automatically.

```
$# Push your code to GitHub to trigger automatic Terraform Cloud plan
 git add .
 git commit -m "add S3 bucket config"
 git push origin main
```

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

+ Always add *.tfstate and *.tfstate.backup to your .gitignore file.
+ Use separate Terraform Cloud workspaces for Dev, Staging, and Production.
+ Mark sensitive variables as 'Sensitive' in Terraform Cloud so values stay hidden.
+ Enable notifications in Terraform Cloud via Slack or Email for failed runs.
+ Run terraform fmt before committing code to keep your files neatly formatted.

Watch Out For

! Committing terraform.tfstate files containing plain-text secrets to GitHub.
! Forgetting to run terraform init after adding the cloud {} block in main.tf.
! Hardcoding cloud credentials directly inside provider configuration blocks.
! Making manual changes in the AWS Console while Terraform manages the resource.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mramanrajsingh